

Arquitectura funcional V2

LoanOps RD - Modulos, flujos y dependencias alineadas al stack oficial

Documento	04 - Arquitectura funcional V2
Version	2.0 - MySQL oficial
Proyecto	LoanOps RD
Stack oficial	React + Vite + TypeScript NestJS + Prisma MySQL 8+
Autor del proyecto	PholioDev

Este documento reemplaza la version anterior donde pudiera aparecer PostgreSQL, Neon o Supabase como base ideal. La linea oficial de construccion queda cerrada en MySQL 8+ con NestJS y Prisma.

Arquitectura Funcional V2 - LoanOps RD

Documento: 04 - Arquitectura funcional

Version: 2.0

Producto: LoanOps RD

Stack alineado: React + Vite + NestJS + Prisma + MySQL

> **Decision oficial de stack:** LoanOps RD se construira con **React + Vite + TypeScript** en frontend, **NestJS + TypeScript** como backend API-first, **Prisma ORM** y **MySQL 8+** como base de datos principal. No se usara PostgreSQL, Neon ni Supabase como base principal en esta fase.

> **Criterio de infraestructura:** se prioriza MySQL porque esta disponible en el entorno actual del proyecto y evita pagar una base de datos adicional. La documentacion queda alineada a esa decision.

1. Proposito del documento

Este documento define como debe funcionar LoanOps RD a nivel de modulos, flujos, dependencias, reglas y responsabilidades internas. La version 2.0 mantiene la arquitectura funcional original, pero alinea las decisiones tecnicas con MySQL y NestJS.

No es un documento de diseno visual ni de SQL profundo. Su funcion es explicar que hace cada modulo, que informacion consume, que resultado genera y como se conecta con el resto del sistema.

2. Principios funcionales

2.1 Multiempresa primero

Cada prestamista funciona como empresa dentro del SaaS. Toda operacion debe estar vinculada a `empresa_id`. Ninguna empresa debe consultar, modificar o exportar datos de otra.

2.2 Operacion diaria primero

El sistema prioriza:

- Cobrar hoy.
- Registrar pagos.
- Registrar visitas.
- Crear prestamos.
- Revisar atrasos.
- Controlar caja.

- Cerrar rutas.
- Generar reportes.

2.3 Consistencia financiera

Un pago no es solo un registro. Debe impactar cuota, prestamo, caja, recibo, reportes y auditoria. El backend NestJS debe manejar este flujo con transacciones Prisma sobre MySQL.

2.4 Pocas decisiones por pantalla

Si una pantalla tiene subprocesos relevantes, se maneja como subvista separada.

3. Capas funcionales

Capa	Funcion	Modulos
Acceso y seguridad	Controlar entrada, sesiones y permisos	Auth, sesiones, roles
Nucleo SaaS	Administrar empresas y sucursales	Empresas, sucursales, planes
Operacion crediticia	Gestionar cartera	Clientes, prestamos, cuotas, pagos
Cobranza de campo	Ejecutar cobros diarios	Cobrar Hoy, rutas, visitas, promesas
Control financiero	Controlar dinero y documentos	Caja, recibos, cierres
Inteligencia y auditoria	Analizar y rastrear	Reportes, auditoria, scoring
Experiencia publica	Captacion y soporte	Landing, tutoriales, legales

4. Modulos funcionales

4.1 Autenticacion

Entradas: email, password, refresh token.

Salidas: sesion, usuario, empresa, permisos.

Impacta: auditoria, sesiones, seguridad.

4.2 Empresas

Representa cada tenant.

Impacta todos los modulos operativos.

Regla: empresa suspendida no puede operar nuevas transacciones.

4.3 Sucursales

Segmenta operacion por oficina o zona.

Impacta usuarios, clientes, prestamos, rutas, caja y reportes.

4.4 Usuarios y permisos

Define quien puede operar.

El backend debe validar permisos aunque la UI oculte botones.

4.5 Clientes

Gestiona deudores.

Impacta prestamos, rutas, pagos, scoring y reportes.

4.6 Perfil del cliente

Consolida datos, prestamos, cuotas, pagos, visitas, promesas, fichas y documentos.

4.7 Prestamos

Gestiona ciclo de vida del prestamo.

Todo prestamo debe generar cuotas.

4.8 Cuotas

Calendario financiero.

Alimenta Cobrar Hoy, mora, pagos y reportes.

4.9 Pagos

Registra cobros.

Debe ejecutarse en transaccion y afectar cuota, prestamo, caja, recibo y auditoria.

4.10 Cobrar Hoy

Pantalla principal del cobrador.

Muestra clientes con cuotas del dia, atrasos, promesas y acciones rapidas.

4.11 Rutas

Organiza trabajo diario de cobradores.

Una ruta cerrada no se edita libremente.

4.12 Caja

Controla entradas, salidas y cierres.

Todo pago genera entrada de caja.

4.13 Reportes

Convierte datos operativos en indicadores financieros y operativos.

4.14 Configuración

Define reglas por empresa: mora, recibos, prestamos, branding y notificaciones.

4.15 Auditoria

Registra acciones críticas.

No debe ser editable desde UI.

5. Dependencias funcionales

Modulo	Depende de	Impacta
Cientes	Empresa, sucursal, usuario	Prestamos, rutas, reportes
Prestamos	Cliente, configuracion	Cuotas, pagos, reportes
Cuotas	Prestamo	Cobrar Hoy, pagos, mora
Pagos	Cliente, prestamo, cuota	Caja, recibos, auditoria
Rutas	Cobrador, clientes, cuotas	Visitas, pagos, reportes
Caja	Pagos, movimientos	Cierres, reportes
Reportes	Todos los modulos	Decisiones
Auditoria	Eventos criticos	Seguridad y control

6. Flujos principales

6.1 Crear cliente

1. Usuario entra a Cientes.
2. Completa datos.
3. Backend valida empresa, sucursal y cobrador.
4. Guarda cliente con empresa_id.
5. Registra auditoria.

6.2 Crear prestamo

1. Usuario selecciona cliente.
2. Backend valida que no este bloqueado.
3. Usuario define monto, interes, plazo y frecuencia.
4. Backend recalcula totales.
5. Backend crea prestamo y cuotas en transaccion.
6. Registra auditoria.

6.3 Registrar pago

1. Usuario registra monto.
2. Backend valida saldo.
3. Prisma inicia transaccion MySQL.
4. Crea pago.
5. Actualiza cuota.
6. Actualiza prestamo.
7. Crea caja.
8. Crea recibo.
9. Registra auditoria.

6.4 Cerrar ruta

1. Usuario revisa resultados.
2. Sistema resume esperado, cobrado y pendientes.
3. Usuario confirma.
4. Backend marca ruta cerrada.
5. Registra auditoria.

6.5 Cerrar caja

1. Usuario revisa balance teorico.
2. Ingresa monto real.
3. Sistema calcula diferencia.
4. Usuario confirma.
5. Backend registra cierre y auditoria.

7. Eventos funcionales

Evento	Disparador	Impacto
cliente.creado	Guardar cliente	Perfil, auditoria
prestamo.creado	Confirmar wizard	Cuotas, perfil, auditoria
cuota.vencida	Job diario	Cobrar Hoy, reportes
pago.registrado	Confirmar pago	Cuota, prestamo, caja, recibo
pago.anulado	Correccion	Reverso caja, auditoria
visita.registrada	Cobrador	Historial, ruta
promesa.vencida	Job diario	Alertas, scoring
ruta.cerrada	Confirmacion	Reportes, auditoria
caja.cerrada	Confirmacion	Reportes, auditoria

8. Matriz funcional por rol

Modulo	Super Admin	Admin Empresa	Supervisor	Cobrador
Empresas	Total	No	No	No
Usuarios	Global	Empresa	Equipo	No
Clientes	Soporte auditado	Total	Segun permiso	Asignados
Prestamos	Soporte auditado	Total	Ver/aprobar	Asignados
Pagos	Auditoria	Registrar/anular	Supervisar	Registrar
Cobrar Hoy	No operativo	Supervisar	Supervisar	Operar
Rutas	Global	Gestionar	Gestionar	Ejecutar
Caja	Global	Cerrar	Ver/cerrar segun permiso	Pagos asociados
Reportes	Global	Empresa	Operativos	Limitados
Configuracion	Sistema	Empresa	Limitado	No

9. Arquitectura funcional web

El panel web se usa para administracion, supervision y control.

Patron de pantalla:

1. Header con titulo y descripcion.
2. KPIs si aplica.
3. Filtros visibles.
4. Tabla/cards.
5. Accion primaria.

6. Estados vacios.
7. Confirmaciones para acciones criticas.

10. Arquitectura funcional movil

La app movil es para el cobrador.

Prioridades:

- Cobrar Hoy.
- Registrar pago.
- Registrar visita.
- Promesa.
- Ruta.
- Mapa.
- Recibo.
- Sincronizacion offline futura.

11. Alineacion tecnica

La arquitectura funcional debe implementarse con:

- Frontend React + Vite.
- Backend NestJS.
- Prisma como capa de datos.
- MySQL como base oficial.
- Transacciones para operaciones financieras.
- Guards para roles y permisos.
- Jobs internos para mora y promesas.

12. Checklist funcional QA

- Ninguna empresa ve datos de otra.
- Cliente bloqueado no recibe prestamo.
- Pago actualiza cuota, prestamo, caja, recibo y auditoria.
- Ruta cerrada no se edita libremente.
- Caja cerrada no recibe movimientos sin permiso.
- Cobrar Hoy muestra solo clientes correspondientes.

- Reportes respetan empresa, sucursal, cobrador y fecha.
- Backend valida todo aunque frontend oculte botones.

13. Conclusion

La arquitectura funcional de LoanOps RD se mantiene como sistema operativo de prestamos y cobranzas, ahora alineado a la ruta tecnica oficial: React + Vite, NestJS, Prisma y MySQL. El criterio central sigue siendo el mismo: operacion clara, trazabilidad completa e integridad financiera.